COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	09/08/2026	**DATE SUBMITTED:**	09/16/2026
NAME:	Dela Cruz, Jared Isaiah P.	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

HANDS-ON ACTIVITY NO. 7.1	**SORTING ALGORITHMS PT1**

1. PROCEDURE
OUTPUT

ILO A: Create C++ code for implementation of selection, bubble, insertion and merge sort.

Preparation Task:

- Create an array of elements with random values. The array must contain 100 elements that are not sorted. Use the created array for each sorting algorithm below. Show the output of the preparation task on table 7-1 in section 6.
- Create a header file for the implementation of the different sorting algorithms.
- Import this header file into your main.cpp.

A.1. Bubble Sort Technique

Using the bubble sort technique, sorting is done in passes or iteration. Thus at the end of each iteration, the heaviest element is placed at its proper place in the list. In other words, the largest element in the list bubbles up.

General Algorithm

```
template <typename T>
void bubbleSort(T arr[], size_t arrSize){
    //Step 1: For i = 0 to N-1 repeat
    Step 2 for(int i = 0; i < arrSize;
    i++){
        //Step 2: For J = i + 1 to N - I repeat
        for(int j = i+1; j < arrSize; j++){

            //Step 3: if A[J] >
            A[i]
            if(arr[j]>arr[i]){
                //Swap A[J] and
                A[i]
                std::swap(arr[j
                ], arr[i]);
            }
            //[End of Inner for loop]
```

```
            }
            //[End if Outer for loop]
        }
        //Step 4: Exit
    }
```

Include a screenshot of your code and output console along with your observations in table 7-2.

A.2.Selection Sort Algorithm

Selection sort is quite a straightforward sorting technique as the technique only involves finding the smallest element in every pass and placing it in the correct position. Selection sort works efficiently when the list to be sorted is of small size but its performance is affected badly as the list to be sorted grows in size.
Hence we can say that selection sort is not advisable for larger lists of data.

```
template <typename T> int Routine_Smallest(T A[], int K, const int
arrSize);
```

//General Algorithm
//Selection Sort (Array, Size of Array)

```
template <typename T>
void selectionSort(T arr[], const int N){ int POS, temp, pass=0;
//Step 1: Repeat Steps 2 and 3 for K = 1 to N-1 for(int i = 0; i <
N; i++){
//Step 2: Call routine smallest(A, K, N,POS) POS =
Routine_Smallest(arr, i, N);
temp = arr[i];
//Step 3: Swap A[K] with A [POS] arr[i] = arr[POS];
arr[POS] = temp;
                //Count pass++;
}
//[End of loop]
//Step 4: EXIT
}
```

//Routine smallest (Array, Current Position, Size of Array)

```
template <typename T>
int Routine_Smallest(T A[], int K, const int arrSize){ int position, j;
//Step 1: [initialize] set smallestElem = A[K] T smallestElem = A[K];
//Step 2: [initialize] set POS = K position = K;
//Step 3: for J = K+1 to N -1,repeat for(int J=K+1; J < arrSize; J++){
if(A[J] < smallestElem){ smallestElem = A[J];

position = J;
}
}
//Step 4: return POS return position;
}
```

Include a screenshot of your code and output console along with your observations in table 7-3.

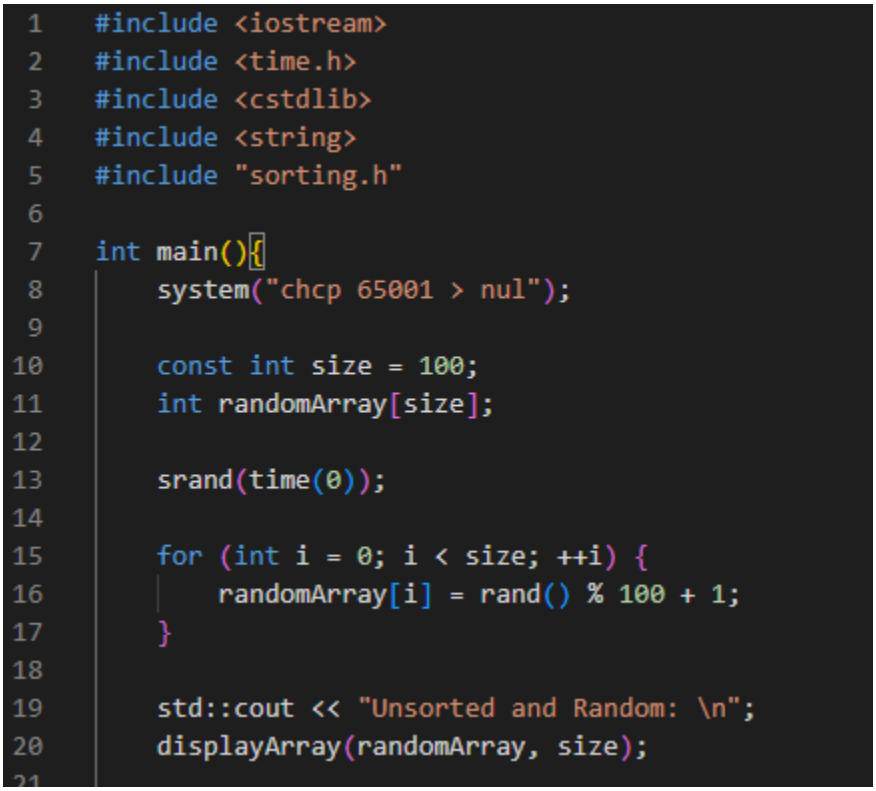
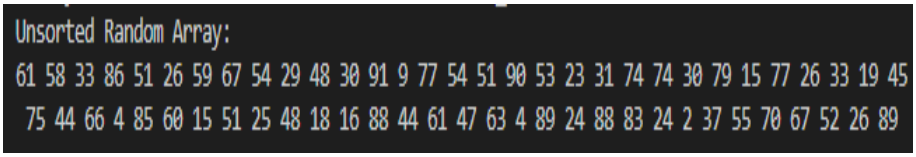
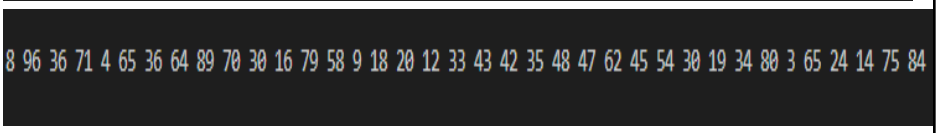
A.2.Insertion Sort Algorithm

In the insertion sort technique, we start from the second element and compare it with the first element and put it in a proper place. Then we perform this process for the subsequent elements. We compare each element with all its previous elements and put or insert the element in its proper position. Insertion sort technique is more feasible for arrays with a smaller number of elements. It is also useful for sorting linked lists. Linked lists have a pointer to the next element (in case of a singly linked list) and a pointer to the previous element as well (in case of a doubly linked list). Hence it becomes easier to implement insertion sort for a linked list.

General Algorithm

```
//General Algorithm
//Insertion Sort template <typename T>
void insertionSort(T arr[], const int N){ int K = 0, J, temp;
//Step 1: Repeat Steps 2 to 5 for K = 1 to N-1 while(K < N){
//Step 2: set temp = A[K] temp = arr[K];
//Step 3: set J = K - 1 J = K-1;
//Step 4: Repeat while temp <=A[J] while(temp <= arr[J]){
//set A[J + 1] = A[J] arr[J+1] = arr[J];
                    //set J = J - 1 J--;
//[end of inner loop]
}
//Step 5: set A[J + 1] = temp arr[J+1] = temp;
//[end of loop] K++;
}
//Step 6: exit
}
```

Include a screenshot of your code and output console along with your observations in table 7-4.

Code + Console Screenshot	  
Observations	This code set up creates an array of 100 integers, populates it with random numbers between 1 and 100 using srand(time(0)) and rand(), and then prints the unsorted numbers to the screen using a custom displayArray(randomArray, size).

```
#include <iostream>
#include <time.h>
#include <cstdlib>
#include <string>
#include "sorting.h"

int main(){
    system("chcp 65001 > nul");

    const int size = 100;
    int randomArray[size];

    srand(time(0));

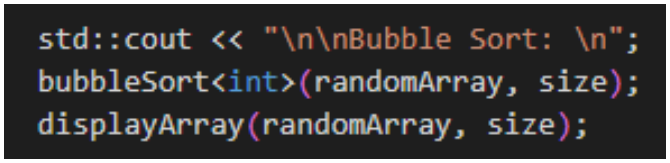
    for (int i = 0; i < size; ++i) {
        randomArray[i] = rand() % 100 + 1;
    }

    std::cout << "Unsorted and Random: \n";
    displayArray(randomArray, size);
```

```
Unsorted Random Array:
61 58 33 86 51 26 59 67 54 29 48 30 91 9 77 54 51 90 53 23 31 74 74 30 79 15 77 26 33 19 45
 75 44 66 4 85 60 15 51 25 48 18 16 88 44 61 47 63 4 89 24 88 83 24 2 37 55 70 67 52 26 89
```

```
8 96 36 71 4 65 36 64 89 70 30 16 79 58 9 18 20 12 33 43 42 35 48 47 62 45 54 30 19 34 80 3 65 24 14 75 84
```

Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	

```
std::cout << "\n\nBubble Sort: \n";
bubbleSort<int>(randomArray, size);
displayArray(randomArray, size);
```

<table>
<tr>
<td></td>
<td>

```
template <typename T>
void bubbleSort(T arr[], int maxSize){
    for(int i = 0; i < maxSize; i++){
        for(int j = i + 1; j < maxSize; j++){
            if(arr[i] > arr[j]){
                std::swap(arr[i], arr[j]);
            }
        }
    }
}
```

```
Bubble Sort:
2 3 4 4 4 8 9 9 12 14 15 15 16 16 18 18 19 19 20 23 24 24 24 24 25 26 26 26 29 30 30 30 30
 64 65 65 66 67 67 70 70 71 74 74 75 75 77 77 79 79 80 83 84 85 86 88 88 89 89 89 90 91 96
```

```
31 33 33 33 34 35 36 36 37 42 43 44 44 45 45 47 47 48 48 48 51 51 51 52 53 54 54 54 55 58
```

```
58 59 60 61 61 62 63
```

</td>
</tr>
<tr>
<td>Observations</td>
<td>In the program, it showcases the bubbleSort() function. It sorts the array by comparing each element and proceeds to swap them when they are in the wrong position or order. During this, it repeatedly checks the elements in the array until they are in ascending order. It compares each arr[i] with the elements adjacent to it.</td>
</tr>
</table>

Table 7-2. Bubble Sort Technique

Code + Console Screenshot

```
std::cout << "\n\nSelection Sort: \n";
selectionSort<int>(randomArray, size);
displayArray(randomArray, size);
```

```
template <typename T>
void selectionSort(T arr[], int arrSize){
    int min;

    for(int i = 0; i < arrSize; i++){
        min = routine_smallest(arr, i, arrSize);
        std::swap(arr[i], arr[min]);
    }

}
```

```
template <typename T>
int routine_smallest(T arr[], int k, int arrSize){
    int position = k;

    T smallestElement = arr[k];

    for(int j = k + 1; j < arrSize; j++){
        if(arr[j] < smallestElement){
            smallestElement = arr[j];
            position = j;
        }
    }
    return position;
}
```

```
Selection Sort:
2 3 4 4 4 8 9 9 12 14 15 15 16 16 18 18 19 19 20 23 24 24 24 24 25 26 26 26 29 30 30 30 3
 64 65 65 66 67 67 70 70 71 74 74 75 75 77 77 79 79 80 83 84 85 86 88 88 89 89 89 90 91 96

31 33 33 33 34 35 36 36 37 42 43 44 44 45 45 47 47 48 48 48 51 51 51 52 53 54 54 54 55 58 58 59 60 61 61 62 63
```

Observations	As I observe this code, it is clear that it implements a template-based Since for this I showcased the Selection Sort algorithm using a helper function `routine_smallest()` to find the index of the minimum value in the unsorted portion of the array, swapping it into place with `std::swap()` at each step, and then displaying the final sorted array.

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot	

```
std::cout << "\n\nInsertion Sort: \n";
insertionSort<int>(randomArray, size);
displayArray(randomArray, size);
```

```
template <typename T>
void insertionSort(T arr[], int arrSize){
    int k = 1, j;
    T temp;

    while(k < arrSize){
        temp = arr[k];
        j = k - 1;
        while(j >= 0 && temp <= arr[j]){
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = temp;
        k++;
    }
}
#endif
```

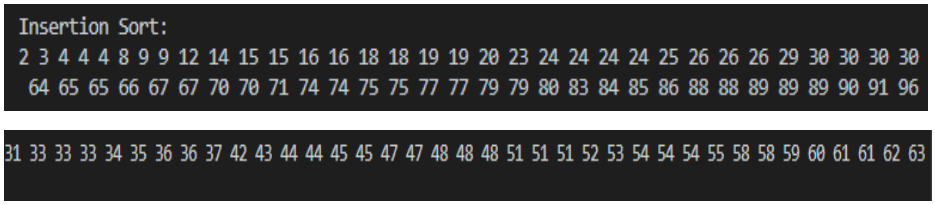
	
Observations	For this line of code I showcase the InsertionSort() function. It picks each number one by one, then proceeds to shift larger numbers to the right to make room, inserts the number into its correct position, and then prints the sorted list.

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Candidate 1	Bo Dalton Capistrano
Candidate 2	Cornelius Raymon Agustín
Candidate 3	Deja Jayla Bañaga
Candidate 4	Lalla Brielle Yabut
Candidate 5	Franklin Relano Castro

List of Candidates

Problem: Generate an array A[0…100] of unsorted elements, wherein the values in the array are indicative of a vote to a candidate. This means that the values in your array must only range from 1 to 5. Using sorting and searching techniques, develop an algorithm that will count the votes and indicate the winning candidate.

NOTE: The sorting techniques you have the option of using in this activity can be

either bubble, selection, or insertion sort. Justify why you chose to use this sorting algorithm.

Deliverables:

- Pseudocode of Algorithm
- Screenshot of Algorithm Code
- Output Testing

Pseudocode of Algorithm

```
START

CREATE an array A with 100 elements.
GENERATE random values from 1 to 5 and store them in A.
SORT array A USING Insertion Sort.
INITIALIZE:
    count = 0
    currentVote = A[0]
FOR i = 0 TO 99
    IF A[i] == currentVote
        count = count + 1
    ELSE
        Display currentVote and count
        currentVote = A[i]
        count = 1
    END IF
  END FOR
DISPLAY the last candidate's vote count.
COMPARE the vote counts of Candidates 1-5.
The candidate with the highest number of votes is declared
the winner.
END
```

Code Screenshots

```
const int voteSize = 100;
int votes[voteSize];

std::string candidates[5] = {
    "Bo Dalton Capistrano",
    "Cornelius Raymon Agustín",
    "Deja Jayla Bañaga",
    "Lalla Brielle Yabut",
    "Franklin Relano Castro"
};

srand(12345);

for (int i = 0; i < voteSize; i++) {
    votes[i] = rand() % 5 + 1;
}

std::cout << "\n\nUnsorted Votes:\n";
displayArray(votes, voteSize);
std::cout << "\n\n";

insertionSort(votes, voteSize);
```

```
std::cout << "Sorted Votes:\n";
displayArray(votes, voteSize);
std::cout << "\n\n";

int voteCount[5] = {0};

for (int i = 0; i < voteSize; i++) {
    voteCount[votes[i] - 1]++;
}

std::cout << "===== ELECTION VOTING RESULTS =====\n";

int winner = 0;

for (int i = 0; i < 5; i++) {
    std::cout << "Candidate " << i + 1 << ": "
              << candidates[i] << " - "
              << voteCount[i] << " votes\n";

    if (voteCount[i] > voteCount[winner]) {
        winner = i;
    }
}
```

```
std::cout << "\nWINNER:\n";
std::cout << candidates[winner]
          << " with " << voteCount[winner]
          << " votes.\n";
```

Screenshot of Output

```
Sorted Votes:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
```

Manual Count

Candidate 1: Bo Dalton Capistrano - 24 votes
Candidate 2: Cornelius Raymon Agustín - 15 votes
Candidate 3: Deja Jayla Bañaga - 22 votes
Candidate 4: Lalla Brielle Yabut - 18 votes
Candidate 5: Franklin Relano Castro - 21 votes

Count Result of Algorithm

```
===== ELECTION VOTING RESULTS =====
Candidate 1: Bo Dalton Capistrano - 24 votes
Candidate 2: Cornelius Raymon Agustín - 15 votes
Candidate 3: Deja Jayla Bañaga - 22 votes
Candidate 4: Lalla Brielle Yabut - 18 votes
Candidate 5: Franklin Relano Castro - 21 votes

WINNER:
Bo Dalton Capistrano with 24 votes.
```

Question: Was your developed vote counting algorithm effective? Why or why not?

- I think the vote counting algorithm is effective since it proved successful as it efficiently organized the given votes, tallied the votes for every candidate, and determined the candidate with the maximum vote count. Employing insertion sort enabled the disorganized vote array to be organized methodically, then the values are effectively tailed according to their related candidate numbers.

COMPUTER ENGINEERING

QUEZON CITY

3. CONCLUSIONS

Through this activity, I learned all about the capabilities of Sorting Algorithms in its entirety. Generating a list of 100 random numbers, displaying them, and then organizing the list using two different sorting methods, which are the Selection Sort and Insertion Sort, help further my knowledge about these algorithms. By printing the array before and after, it shows procedurally how these algorithms take a random set of numbers and put them in specified orders.

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT and Google Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **AI is used for grammar checking and program debugging/verification**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The usage of AI did not replace any of my personal contributions toward this activity. Aside from the stated use of AI, everything else is of my own creation.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.